Week 25 (Days 169-175)

Status: Days 169-175 are written in full. This is the start of the expert-level final capstone: a CPU-feasible on-device-style **Course Assistant** that integrates the whole course - corpus indexing, MiniLM retrieval, SmolLM2-135M generation (optional LoRA), a tool-using agent with injection guards, a Rust/PyO3 int8 scoring kernel, an evaluation harness with regression gates and PSI drift, a responsible-AI audit with a model card, and a tested stdlib HTTP service. Every number comes from `code/week-25/day169_*.py` ... `day175_*.py` and the shared package under `code/capstone/`, each run on this box, with logs in `code/week-25/out/`. Shared helpers live in `code/week-25/_dep.py`.

Retitled from the original SYLLABUS rows. Row 169 was "Governance, regulation, careers; reading papers efficiently" (covered in spirit by Week 24 Day 167's NIST/EU overview and model cards). Rows 170-175 were generic FINAL CAPSTONE phase labels. They are retitled to match this week's build milestones: proposal/budget -> data/model -> RAG+agent -> performance -> eval harness -> safety/audit -> integration + Month-6 midpoint exam.

The system under build. Corpus = `weeks/*.md` (26 files -> **1467** chunks, **1,075,590** chars). Retriever = `sentence-transformers/all-MiniLM-L6-v2` (384-d). Generator = `HuggingFaceTB/SmolLM2-135M-Instruct` (**134,515,008** params). Optional adapter = LoRA r=8 on `q_proj`/`v_proj` (`code/capstone/adapters/course_lora`). Tools = allow-listed `calc` (AST-safe), `course_search`, `get_day_title`. Scoring kernel = `code/capstone/rust_score` (PyO3 + rayon int8). Service = `code/capstone/service.py` (stdlib `HTTPServer` on `:8765`).

Compute note. CPU-only 8-core Xeon, ~15 GB RAM, no GPU. HF weights live under `data/cache/hf`. A GPU would shrink decode latency but would not change recall, gates, or the RAI artefacts. Numbers below are from this box only; do not treat them as phone or laptop claims.

Honesty labels. LoRA-only generations after 60 steps are **nonsense** (logged Day 170) - that is a teaching result that RAG is the quality path on this budget. OpenBLAS fp32 can beat Rust int8 on the tiny 1467x384 corpus; the >=1.5x gate is measured against a **numpy-int8** baseline and on a **50kx384** synthetic matrix (Day 172). Groundedness uses chunk-id / word-overlap heuristics; some fluent answers still mis-state facts (e.g. k-means) - the gate measures grounding, not perfect factuality.

Day 169: Capstone proposal, success metrics and budget table

Review today: Day 168 (+1), Day 165 (+3), Day 161 (+7), Day 148 (+21; quick skim of the Month-5 RAG/agent self-check).

Block 1: What "done" means for an end-to-end assistant (30 min)

A capstone that only trains a model is not done. Done means: a **spec** with measurable success metrics, a **budget table** that fits the hardware, a **package** others can run, and an explicit **non-goal** list. Today's deliverable is that spec. The rest of the week implements it.

Non-goals for Week 25: no web browsing tool, no GPU training, no claim of production readiness, no personal-data corpus, no "own tokenizer from scratch" as a hard requirement (optional stretch - MiniLM's tokenizer is used for embeddings; SmolLM2's for generation).

Block 2: Run the proposal script (30 min)

```
cd code/week-25 && python day169_proposal.py
```

The script chunks all week files, times a 64-chunk MiniLM embed, loads SmolLM2, and writes `out/day169_budget.json` plus `code/capstone/out/budget.json`.

Verified (`out/day169_proposal.log`): - Corpus: **26** week files -> **1467** chunks; **1,075,590** total chunk chars. - Embed 64 chunks: **0.605 s** (**9.4 ms/chunk**); dim 384. Full-index estimate ~= **13.9 s**. - SmolLM2 params: **134,515,008**. Generate ~38 tokens: **0.524 s** (**13.8 ms/tok**).

Block 3: Success metrics and budget table (35 min)

Copy these into your own design doc; they gate Days 173 and 175.

Metric	Target
retrieval recall@3	>= 0.70 on hand-labelled QA
grounded answer rate	>= 0.60
tool success	>= 0.80 on allow-listed tasks
injection block	>= 0.80 (0 FP on 10 benign)
p50 RAG+gen	<= 8.0 s (40 new toks, CPU)
embed index build	<= 120 s
LoRA train	<= 15 min
int8 score vs numpy-int8	>= 1.5x (large matrix / fair baseline)
release gate	FAIL if recall drops >10% relative or injection < 0.70
RAI	model card + audit filed; no real-user deployment claim

Budget rows: RAM growth <= 4 GiB for MiniLM+SmolLM2; LoRA r<=8 on q_proj/v_proj; max_new_tokens 40 default / 80 hard; top_k=3; tools = calc, course_search, get_day_title.

Block 4: Self-check (20 min)

1. Why is p50 <= 8 s a *latency* budget and not an accuracy one? *It bounds CPU UX; accuracy has its own recall/grounded rows.*
2. Why list non-goals? *Prevents scope creep into browsing agents or GPU-only tricks that the box cannot verify.*
3. What fails a release even if recall is 0.90? *Injection block < 0.70, or a >10% relative recall drop vs baseline.*
4. How many chunks does the proposal script report? ***1467.***

Block 5: Practice (30 min)

Write a one-page design doc of your own: architecture diagram (boxes for embed / index / retrieve / generate / tools / guards / eval), the table above, and three risks with mitigations. Save it next to `code/capstone/README.md`.

Extra notes

- The "20-question" metric row was written before the final QA list landed at **25** pairs; Day 170+ use all 25. Keep the >=0.70 threshold.
- `embed_full_est_s` 13.9 was an extrapolation from 64 chunks; Day 170 measures the real full pass at **16.11 s**.

Going deeper

Re-read Week 21's agent gates and Week 22's LLMOps regression ideas. Capstone gates are those ideas applied to one product.

Interview-style question (15 min)

"We will ship when the demo looks good." Reply: demos lie. Ship when recall@3, grounded rate, tool rate, injection block, and p50 latency all clear the budget table on a frozen eval set, with a written model card.

Common mistakes

- Optimising generation style before measuring retrieval recall.
- Adding tools without an allow-list and AST/schema guard.
- Claiming "on-device" without a memory/KV budget (Day 172).

Build tasks (student)

1. Fork the budget JSON and change one latency number; justify the change in one paragraph tied to your CPU model.
2. Sketch the sequence diagram for one `/ask` call: guard -> route -> tool or retrieve -> generate -> scorecard fields touched.
3. List three metrics you will *not* optimise this week (e.g. BLEU, human Likert, GPU tokens/sec) and why they are out of scope on this box.

A good proposal is boring on purpose. It names the corpus path, the exact Hugging Face model ids, the top_k, the max_new_tokens, and the freeze-set size. It also names what happens when a gate fails: the answer is "do not ship", not "add a

disclaimer in the README". Week 22's LLMOps lesson applies unchanged - regression gates are product code.

Hardware honesty matters in Month 6. This box is an 8-core Xeon with about 15 GB of RAM. SmolLM2-135M in fp32 is roughly half a gibibyte of weights; MiniLM is tiny beside it; the dangerous growth terms are the KV cache (Day 172) and Python allocator fragmentation under repeated generate calls. If you later move the same budget table to a phone, every millisecond and every mebibyte must be re-measured on that phone. Copying server-CPU numbers into an "on-device" slide is a integrity failure, not a documentation shortcut.

The success-metric table mixes quality (recall, grounded, tools, injection) with performance (p50, index build, LoRA minutes, int8 speedup) and process (RAI artefacts). Mixing them in one table is intentional: a fast wrong system fails, and a correct un-audited system also fails. When two metrics conflict - for example, raising top_k to help recall while hurting p50 - write the trade-off in the design doc instead of silently moving the goalposts.

Finally, freeze the eval pairs before you tune prompts. The 25 QA rows in `corpus_qa.py` are small enough to memorise; if you edit prompts while staring at them, you no longer have an eval set. Day 173's relative-drop gate only works if Day 171's scorecard stays immutable on disk.

What to do next

Day 170 builds the index and a LoRA baseline against the budget's train-time row.

Day 170: Data, corpus index and LoRA baseline

Review today: Day 169 (+1), Day 166 (+3), Day 162 (+7), Day 149 (+21).

Block 1: Chunking and labelling (30 min)

Retrieval quality starts with chunks, not with the LM. `_dep.chunk_weeks` splits each `week-XX.md` on `##` headers, then packs into <=800-char pieces with ids like `week-14.md#4-0`. Labels for eval are **week file substrings** on 25 hand-written pairs in `code/capstone/corpus_qa.py` (gradient descent -> week-01, RAG -> week-17, demographic parity -> week-24, ...).

Block 2: Build the index and train LoRA (40 min)

```
python day170_data_model.py
```

Verified (`out/day170_data_model.log`, `day170_stats.json`): - Chunks: **1467**. Full embed: **16.11 s** for 1467x384; L2-normalised; `embeddings.npy` ~= **2.3 MB**; `chunks.json` saved under `code/capstone/out/`. - Retrieval recall@3 by week-tag: **18/25 = 0.720** (clears >=0.70). - LoRA: 60 steps in **14.8 s** (**0.25 min** ? 15 min budget). Loss **3.791 -> 0.961** (logged end); intermediate steps 20/40/60: 2.957 / 1.504 / 1.351. - Adapter saved to `code/capstone/adapters/course_lora`.

Block 3: Why LoRA-alone fails here (35 min)

Ask the LoRA model "What is RAG?" without retrieval. Logged answer: *"A recursive algorithm for finding a path from a root to a leaf in a tree."* Demographic parity becomes sibling-counting nonsense. Teaching point: on a 135M base with 60 CPU steps and no label masking of the prompt, **adapters do not replace RAG**. Day 171 makes retrieval mandatory in the answer path.

Block 4: Self-check (20 min)

1. Is 16.11 s under the embed budget? *Yes (<=120 s).*
2. Recall@3 = 0.720 - pass or fail? *Pass (>=0.70).*
3. Why is a low training loss not enough? *The generations are still factually wrong without context.*
4. Where do embeddings live? *`code/capstone/out/embeddings.npy`.*

Block 5: Practice (30 min)

Add two more QA pairs for a week that scored a miss in the 7/25 failures. Re-run only the recall loop (or the full Day 170 script) and record the new fraction. Do not tune on the test pairs if you later promote them into the freeze set.

Extra notes

- peft 0.21.x LoRA r=8 on q_proj/v_proj keeps the adapter small.
- Full-precision SmolLM2 weights stay in HF cache; only adapters are under `code/capstone/adapters/`.

Going deeper

Mask prompt tokens in the LoRA loss (labels = -100 on the instruction span) and compare a 60-step run. Expect better parroting of answers, still weak open-ended behaviour - then return to RAG.

Interview-style question (15 min)

"Loss went from 3.8 to 0.96 - we are done fine-tuning." Reply: show the RAG and demographic-parity generations. Loss without a frozen behavioural eval is vanity.

Common mistakes

- Embedding without L2 normalisation then using dot-product as cosine.
- Training LoRA on GPU assumptions you cannot verify on this box.
- Deleting the 7 miss cases instead of analysing chunk boundaries.

Build tasks (student)

1. Print five random chunks; check that no chunk mid-cuts a code fence in a way that makes the text unreadable. Adjust `max_chars` if needed and re-embed.
2. Compute recall@1 and recall@5 on the same 25 pairs; record all three. Does @5 saturate?
3. Open one LoRA adapter weight file size (`du -h adapters/course_lora`); confirm it is far smaller than the base model.

Chunk identity (`week-14.md#4-0`) is part of the groundedness story later. If you re-chunk with different parameters, every id changes and Day 171's "cite chunk ids" instruction becomes a moving target. Treat chunking parameters as frozen once the index is built for the freeze-set run - or version the index directory (`out/index-v1/`).

Why week-tag recall instead of exact answer match? Because the generator is a 135M instruct model on CPU with 40 new tokens. Exact-match answer grading would mostly measure generation luck. Retrieving the right week is a property of the embedding index and the question wording, which is what you can actually improve this week with chunking and query phrasing.

The LoRA run is deliberately short. Sixty steps on 25 short pairs is enough to drop training loss and to produce confident wrong answers. That combination - low loss, wrong answers - is the pedagogical payload. Students who stop at the loss curve ship a regression. Students who read the two sample generations keep RAG in the critical path and treat LoRA as an optional flavour.

If you insist on improving LoRA: mask the instruction tokens, raise steps carefully inside the 15-minute budget, and still evaluate with the Day 173 harness under `use_lora=True`. Do not replace the RAG path in `Agent.run` until the harness says the LoRA path matches grounded >=0.60 on the freeze set - which it did not on Day 170's bare generations.

Timed 6-hour schedule (suggested)

- 0:00-0:20 Review (+1/+3/+7/+21) and skim `corpus_qa.py`.
- 0:20-1:00 Block 1-2: chunk + embed + recall.
- 1:00-1:40 LoRA train and read the bad generations aloud.
- 1:40-2:20 Block 3 write-up in your notes: "loss != behaviour".
- 2:20-3:00 Self-check + practice recall@1/@5.
- 3:00-4:00 Going deeper optional mask-loss experiment (time-box).
- 4:00-5:00 Interview answer + common mistakes journal.
- 5:00-6:00 Commit artefacts: embeddings.npy, chunks.json, adapters/, day170_stats.json, log.

The index artefacts are the capstone's real "model weights" for retrieval. Lose them and Days 171-175 become expensive. Prefer checking `embeddings.npy` shape `(1467, 384)` after every re-embed. If the shape changes, every downstream p50 and recall number must be re-frozen.

Data hygiene: week files still being edited elsewhere in the course will drift the corpus. For Week 25, treat `weeks/week-01.md` ... `week-26.md` as the corpus inputs; rebuild the index if any week file changes. If you must fix a typo in a week file, rebuild the index and re-run Days 170-173 as a chain - never mix an old `embeddings.npy` with new `chunks.json`.

What to do next

Day 171 wires retrieve -> generate and the tool agent with guards.

Day 171: RAG retrieval and tool-using agent

Review today: Day 170 (+1), Day 167 (+3), Day 163 (+7), Day 150 (+21).

Block 1: Grounded generation pattern (30 min)

`CourseRAG.answer` retrieves top-3, builds a chat template that says *Answer using ONLY the context. Cite chunk ids.*, then greedily generates <=40 new tokens with SmolLM2. Groundedness heuristic: chunk id / week stub appears in the answer, or (>=2) overlapping content words with the context.

Block 2: Run RAG + agent (40 min)

```
python day171_rag_agent.py
```

Verified (`out/day171_rag_agent.log`, `day171_stats.json`): - recall@3: **18/25 = 0.720**; p50 retrieve **118.3 ms**. - Grounded on 8 questions: **8/8 = 1.000**; p50 RAG+gen **0.78 s** (? 8 s). - Agent tools: `calc: (2+3)*4` -> **20**; day-155 title returned; RAG path answered "What is RAG?" without blocking. Tool/agent success **3/3**. - Injection: blocked **3/3** attacks; false positives **0/10** benign.

Block 3: Read the failure modes in the successes (35 min)

Several grounded=True answers are still factually soft: k-means described as "k-nearest neighbors"; random-forest answer pasted a week status blurb. **Grounding != correctness.** The gate still passes because the budget measures citation/overlap, not a human factuality rubric. Capstone write-ups must say this out loud (Day 174 limitations).

Agent routing is pattern-based (math -> calc; "title"+"day N" -> syllabus map; else course_search + RAG). That is enough for the allow-list demo; it is not a learned router.

Block 4: Self-check (20 min)

1. p50 RAG+gen? ***0.78 s.***
2. Injection FP rate? ***0/10.***
3. Why keep top_k=3? *Budget row; more context blows the 1024-token prompt truncate and CPU decode time.*
4. What does `guard_user` return on "Ignore previous instructions..."? *blocked, reason injection pattern.*

Block 5: Practice (30 min)

Add a fourth tool `list_weeks` that returns the sorted week filenames (read-only). Extend `TOOLS` and a routing branch. Confirm injection still blocks before any tool runs.

Extra notes

- `INJECTION_PAT` is a regex - paraphrases can evade (Week 21 lesson). Defence in depth: allow-list + AST calc + no filesystem tools.
- Day titles are scraped from `## Day N:` headings in `weeks/*.md` at Agent init.

Going deeper

Replace the heuristic grounded flag with a tiny entailment check (e.g. overlap + "cite [id]" regex). Re-measure the 8-question set.

Interview-style question (15 min)

"Our agent has tools and a system prompt - we are safe." Reply: show the injection suite and the AST calc rejects. System prompts are not a control boundary; code is.

Common mistakes

- Calling `eval` on calc expressions.
- Measuring only attack block rate and ignoring false positives on benign study questions.
- Shipping LoRA path as default after Day 170's nonsense generations.

Build tasks (student)

1. Log the three hit ids for each of the 25 questions to a CSV; sort by whether the week-tag matched. Spot patterns in the seven misses.
2. Time `answer(..., use_context=False)` vs `use_context=True` on the same eight questions; compare grounded flags.
3. Feed the agent the string `calc: 2**10` and a blocked injection in the same session; confirm order of checks (guard runs first).

Routing policy is intentionally dumb. Learned routers need data you do not have on Day 171. Pattern routing plus an allow-list gives you a testable control plane: every tool name is known, every argument schema is known, every rejection path returns JSON instead of a stack trace (after Day 174's calc hardening).

Context formatting matters. The template truncates each hit to 400 characters and the whole prompt to 1024 tokens. If your week files have the thesis of a section after character 400, retrieval can be "correct" by week-tag while the generator never sees the sentence that answers the question. That is a chunking/truncation bug, not an LM bug. Fix by moving key definitions earlier in sections or by raising the per-hit character cap inside the latency budget.

Injection testing must include benign study questions. A guard that blocks "ignore previous instructions" but also blocks "What is attention?" is useless for a course assistant. The 0/10 FP result is as important as the 3/3 attack result. When you tighten the regex on Day 174's paraphrases, re-run the benign list every time.

Tool traces belong in the response. Day 175's tests assert on `answer` and `blocked`, but production debugging wants `trace` with tool name, args, and raw tool JSON. Keep traces; strip them only at a user-facing presentation layer if needed.

Timed 6-hour schedule (suggested)

- 0:00-0:20 Review; re-read Day 169 budget rows for grounded and injection.
- 0:20-1:20 Implement or run RAG answer loop; fill the 8-question table by hand from the log.
- 1:20-2:20 Agent tools + injection suite.
- 2:20-3:20 Error analysis on grounded-but-wrong answers (k-means, forest blurb).
- 3:20-4:20 Practice: fourth tool or CSV of hit ids.
- 4:20-5:20 Interview + harden any crash paths you notice.
- 5:20-6:00 Update design doc with measured p50 0.78 s and 3/3 tools.

Notice that retrieve p50 (~118 ms) is a small slice of RAG p50 (~0.78 s). Generation dominates on CPU. That tells you where optimisation time goes on Day 172: shaving int8 microseconds off retrieval will not move the user-visible p50 until decode gets faster (quantised generate, fewer new tokens, smaller context). Keep retrieval correct; spend performance energy on decode and on not blowing the prompt length.

The agent return type is a dict with `blocked`, `answer`, optional `trace`, optional `hits`. Day 175's HTTP layer JSON-encodes that dict directly. Stable keys are part of the API contract - rename carefully and update tests in the same change.

What to do next

Day 172 attacks latency and memory: int8 Rust scoring + KV budget.

Day 172: Int8 scoring, Rust/PyO3 kernel and KV budget

Review today: Day 171 (+1), Day 168 (+3), Day 164 (+7), Day 158 (+21; KV formulas).

Block 1: Why a Rust kernel (30 min)

Python numpy int8 matmul for scoring promotes to int32 and is slow when written naively. A PyO3 module can keep int8 weights, accumulate in i32, dequantize with per-row scales, and parallelise with rayon - the same pattern as Week 23's `tiny_infer`. For the **1467x384** course index, OpenBLAS fp32 GEMM is already ~9 µs; the Rust path must be judged fairly (vs numpy-int8, and on larger matrices).

Block 2: Build and benchmark (40 min)

```
# one-time: maturin develop --release in code/capstone/rust_score
python day172_perf.py
```

Verified (`out/day172_perf.log`, `day172_stats.json`): - Corpus 1467x384: numpy fp32 **8.9 µs**; numpy int8 **393.7 µs**; rust serial **58.7 µs** (**6.71x** vs np-int8); rayon **68.7 µs**. Top-3 agreement **3/3**; max|diff vs fp32| **0.0016**. Int8 bytes **569,196**. - Synthetic **50kx384**: np-fp32 **582.7 µs**; np-int8 **12,548.5 µs**; rust rayon **293.0 µs** -> **42.83x** vs np-int8, **1.99x** vs fp32. - Synthetic **100kx256**: rust rayon **50.35x** vs np-int8. Gate >=1.5x vs numpy-int8: **PASS** (best **50.35x**). - SmolLM2 KV fp16: **23,040 B/tok** (22.5 KiB); ctx 512 -> **11.2 MiB**; ctx 2048 -> **45.0 MiB**. - ru_maxrss after SmolLM2: **692 MiB**. Prefill 5 tok: **26.3 ms**. Decode with KV: **12.1 ms/tok**. No-cache decode **1.6x** slower than KV path on this run.

Block 3: Reading the memory story (35 min)

RSS ~692 MiB is under the "<=4 GiB growth" spirit of the budget (weights + python + embeddings). KV at 22.5 KiB/token explains why long chat histories hurt on-device long before weight size does - same formula as Day 158 (GQA with 3 kv heads x 64 dim x 30 layers x 2 x 2 bytes).

Honesty: on the real 1467-row corpus, ship numpy fp32 for scoring unless you care about the int8 memory footprint (~=0.57 MB vs 2.25 MB fp32). Ship Rust when N grows or when you need a portable kernel beside Python.

Block 4: Self-check (20 min)

1. KV bytes per token fp16? ***23040.***
2. Best rust vs numpy-int8 speedup logged? ***50.35x.***
3. Does rust beat OpenBLAS on 1467x384? *No - fp32 GEMM ~8.9 µs vs rust ~59 µs.*
4. No-cache vs KV slowdown factor? ***1.6x** on this decode test.*

Block 5: Practice (30 min)

Call `build_int8_index` once, then score 100 random queries; plot latency histogram. Confirm top-3 agreement stays >=2/3 on 20 random corpus rows used as queries.

Extra notes

- `rust_score` uses pyo3 0.25 / numpy 0.25 to match Week 23's crate versions in the local cargo cache.
- Re-quantizing every call was a first-bug; `build_int8_index` caches scales + int8 matrix.

Going deeper

Add a blocked int8xint8 GEMM via `ndarray` or a hand-written tiling loop; compare to rayon row-parallel.

Interview-style question (15 min)

"Int8 is always faster." Reply: show the 1467-row numbers. Int8 wins on memory and on fair int8 baselines / large N; BLAS fp32 wins on tiny N.

Common mistakes

- Comparing a re-quantize-every-call kernel to BLAS.
- Forgetting KV when claiming "135M fits on device".
- Reporting ASSUMED phone latency from server-CPU runs.

Build tasks (student)

1. Re-run the 50kx384 benchmark with `parallel=False`; compare to rayon. On how many cores does rayon help?
2. Estimate KV mebibytes for ctx=4096 at fp16 and at int8 KV (halve the bytes); does it still fit your RAM budget beside 692 MiB RSS?
3. Add a unit test that `score_int8` raises if called before `build_int8_index`.

The first Rust version re-quantized the database on every score call and lost to numpy by 100x. Caching the int8 matrix and scales flipped the story. Microbenchmark discipline: warm up, median of many trials, report the comparison baseline explicitly (fp32 BLAS vs int8 numpy vs rust). A single "speedup" number without a baseline is marketing.

KV-cache budgeting is the on-device punchline. Weights for SmolLM2 are fixed; KV grows with batch x sequence x layers x kv_heads x head_dim x 2 (K and V) x dtype bytes. With GQA (3 kv heads, 64 dim, 30 layers, fp16) you pay 23040 bytes for every new token of history. A multi-turn tutor session at 2k context is already ~45 MiB of KV before activations. That is why Day 169's max_new_tokens cap exists and why "just raise context" is not a free product decision.

RSS 692 MiB after load is a process peak via `ru_maxrss`, not a precise weight-only figure. Treat it as a ceiling check against the 4 GiB budget row. If you add ONNX Runtime, another tokenizer, or a second model for reranking, re-measure - do not sum datasheet sizes and hope.

When would you ship Rust scoring for this course corpus? When you embed many more notes (whole PDF libraries), when you need identical scores in a non-Python worker, or when you care about the 0.57 MB int8 footprint on a small device. For 1467 rows in a Python service, numpy fp32 is the pragmatic default; keep the crate green anyway - it is the Week 23 skill check inside the capstone.

What to do next

Day 173 freezes the eval harness and regression gates.

Day 173: Evaluation harness, regression gates and drift

Review today: Day 172 (+1), Day 169 (+3), Day 165 (+7), Day 147 (+21).

Block 1: What a harness must do (30 min)

A harness is not a demo notebook. It loads frozen artefacts, computes the budget metrics, compares to a baseline JSON, applies **pass/fail gates**, and writes a scorecard. Drift (PSI on embedding features) watches whether "production" traffic still looks like the index.

Block 2: Run the harness (40 min)

```
python day173_eval.py
```

Verified (`out/day173_eval.log`, `day173_stats.json`): - recall@3 **18/25 = 0.720**; p50 retrieve **119.0 ms**. - grounded **8/8 = 1.000**; p50 RAG+gen **0.75 s**. - tools **3/3**; injection block **3/3 = 1.000**; FP **0/10**. - PSI dim0 base-vs-prod **0.0957**; PSI L2-norm **0.0017**; synthetic +0.5 shift reported **0.0000** on this binning (percentile bins can absorb a constant shift - treat as a caveat; use PSI on norms / multi-dim in production). - Gates: **PASS**; failed=[]; rel_recall_drop **0.000** vs Day 171 baseline 0.72.

Block 3: Gate design review (35 min)

Gates encoded: recall>=0.70, grounded>=0.60, tool>=0.80, injection>=0.80, fp_benign==0, p50_rag<=8.0, no >10% relative recall drop, injection floor 0.70. Any failure -> status FAIL -> Day 175 release blocked.

Relative drop needs a stored baseline (`day171_stats.json`). Without it, the harness would silently accept a cold start - always commit the baseline JSON.

Block 4: Self-check (20 min)

1. Harness status? ***PASS.***
2. p50 RAG? ***0.75 s.***
3. What does rel_recall_drop 0.000 mean? *Same 0.72 as Day 171 baseline.*
4. Why PSI alone is not enough? *Constant shifts can hide in coarse bins; pair with recall gates.*

Block 5: Practice (30 min)

Deliberately scramble 30% of `chunks.json` texts, rebuild nothing else, re-run recall only, and confirm `no_recall_drop_10pct` fails. Restore the file.

Extra notes

- Groundedness still uses the Day 171 heuristic; do not claim human factuality.
- Tool cases include `calc: 17*19` -> 323.

Going deeper

Add a factuality rubric of 5 questions scored by exact keyword sets from the notes; report separately from groundedness.

Interview-style question (15 min)

"All metrics are green in the notebook." Reply: ask for the scorecard JSON, the baseline hash, and which single gate would have blocked last week's regression.

Common mistakes

- Re-tuning prompts on the freeze set.
- Logging only means without p50 latency.
- No FP measurement for injection guards.

Build tasks (student)

1. Add a JSON schema for the scorecard and validate `day173_stats.json` against it in the script.
2. Compute PSI on three random embedding dimensions; report the max. Set a watch threshold (e.g. 0.2) and document it.
3. Break injection on purpose (comment out `guard_user`), re-run, confirm status FAIL, restore.

Baseline discipline: `day171_stats.json` is the contract. If someone re-runs Day 171 after changing chunking, they must bump a version field or the relative-drop gate becomes meaningless. Prefer copying the baseline to `capstone/out/baseline_freeze.json` and pointing Day 173 at that path.

PSI caveats showed up live: a constant +0.5 shift produced a 0.0 PSI with the percentile-bin implementation because both distributions were remapped into the same empirical bins derived from `expected`. That is a reminder to test your drift detector with a known shift and with a known population change (e.g. replace half the corpus with unrelated text). Prefer monitoring features that affect product quality - retrieval score distributions, hit-week histograms - not only dim-0 of the embedding matrix.

Gates should be boring boolean expressions. Clever adaptive thresholds that "learn" from the last week of traffic will eventually learn to accept regressions. Keep the budget numbers in one JSON file shared with Day 169 so the proposal and the harness cannot disagree.

Log every gate's inputs, not only the FAIL set. When Day 175 prints `gate_recall 0.72`, that number should match `day173_stats.json` bit-for-bit. Divergences mean someone re-ran a partial script or edited JSON by hand.

Timed 6-hour schedule (suggested)

- 0:00-0:30 Review budget table beside Day 171 stats.
- 0:30-1:30 Run harness; save scorecard; diff against Day 171.
- 1:30-2:30 PSI experiments (dim0, norms, known shift).
- 2:30-3:30 Deliberate failure practice (scramble chunks); restore.
- 3:30-4:30 Schema-validate scorecard; document gate booleans.
- 4:30-5:30 Interview prep: explain each gate in one sentence.
- 5:30-6:00 Commit `day173_stats.json` as the freeze input for Day 175.

A harness that cannot fail is theatre. The scramble practice exists so you see FAIL in your terminal at least once. Take a screenshot or keep the failing JSON under `out/day173_fail_example.json` for the Week 26 write-up. Hiring committees and instructors trust people who can show a red gate they fixed.

Connect drift to product: if PSI on hit-week histograms spikes, students may have started asking about material outside the indexed week files. The right response is to extend the corpus and re-freeze, not to widen the PSI threshold until the alert goes quiet.

What to do next

Day 174 runs the RAI audit and writes the model card from these numbers.

Day 174: Responsible-AI audit and model card

Review today: Day 173 (+1), Day 170 (+3), Day 166 (+7), Day 167 (+21).

Block 1: Threat model first (30 min)

Seven threats logged: prompt injection, tool abuse, data exfiltration, hallucination, privacy, fairness (topic skew), supply chain. Each has a mitigation already in code (regex guard, AST calc, no file tools, RAG gate, course-only corpus, per-week recall, pinned HF ids + local cache).

Block 2: Red-team and card (40 min)

```
python day174_safety.py
```

Verified (`out/day174_safety.log`, `day174_stats.json`): - Red-team mitigated **6/6** (3 injection blocks + 2 calc rejects for `__import__`/`open` + 1 more injection). Benign FP **0/3**. - Per-week recall (macro): min **0.00**, max **1.00**, mean **0.633** across 15 week tags (micro recall remains 0.72 - some single-question weeks miss). - Model card: `code/capstone/out/MODEL_CARD.md` + `model_card.json` with eval/perf summaries embedded from Days 172-173.

Block 3: What the card must say (35 min)

Intended use: CPU study helper over course notes - **not** a general chatbot, not medical/legal advice. Limitations copied from logs: SmolLM2 hallucinates without RAG; regex injection guards evade paraphrases; fairness proxy is topic recall only (no demographics in corpus); BLAS may beat int8 on tiny N. Ethical note: cite chunks; refuse jailbreaks; do not log secrets.

Agent hardening today: `call_tool("calc")` catches `ValueError` and returns `calc rejected` instead of crashing the service (needed for Day 175).

Block 4: Self-check (20 min)

1. Red-team mitigated fraction? ***6/6.***
2. Where is the model card? *`code/capstone/out/MODEL_CARD.md`.*
3. Why is macro recall 0.63 with micro 0.72? *Equal weight per week tag; singleton misses hurt macro.*
4. Is this cleared for real users? *No - card says no real-user deployment claim.*

Block 5: Practice (30 min)

Add two paraphrase injection strings that evade `INJECTION_PAT`. Document them under "residual risks" in the card. Optionally tighten the regex - re-run FP on the 10 benign list.

Extra notes

- Week 24's NIST/EU overview still applies; this card is product-specific evidence, not legal advice.
- Supply-chain mitigation is pinning + local cache; checksums are optional homework.

Going deeper

Run the Week 24 membership-inference idea on the LoRA training pairs (expect ~chance on 25 Qs) and cite it under privacy.

Interview-style question (15 min)

"We have a model card PDF from marketing." Reply: cards without measured recall, injection FP, and threat mitigations are brochures. Point at `day173_stats.json` and the red-team table.

Common mistakes

- Hiding LoRA nonsense generations.
- Claiming demographic fairness on a corpus with no demographic labels.
- Leaving calc exceptions uncaught so attackers DoS the service.

Build tasks (student)

1. Expand the red-team list to 15 strings; keep FP measurement on >=10 benign prompts.
2. Fill the "contact" and "version" fields of the model card with your name and a semver bump rule.
3. Write a residual-risk paragraph for paraphrase injections and for macro-recall skew toward well-indexed weeks.

Responsible AI in this course is evidence, not vibe. The audit script prints mitigations that exist in code paths you can grep. If a threat row says "AST allow-list" but `run_calc` still calls `eval` on raw strings with names, the audit is false. Day 174 caught a real gap - uncaught `ValueError` from calc - and fixed `agent.py` so the service cannot be crashed by `calc: open(...)`.

Fairness here is a **proxy**: per-week retrieval recall. There are no demographic attributes in `weeks/*.md`, so demographic parity does not apply. Saying "we checked fairness" without saying *which* fairness is a Week 24 failure mode returning as paperwork. The card's fairness section must keep the word "proxy".

Model cards should be generated from logs, not typed from memory. Embedding Day 172 and Day 173 JSON into `model_card.json` prevents the classic drift where the card says recall 0.8 after the harness moved to 0.72. When you re-run Days 172-174, regenerate the card in the same commit.

Non-deployment is a feature of Week 25. The card's intended-use sentence is a control. If a classmate asks to put this behind a public URL for strangers, the answer is no until Week 26+ hardening, abuse rates, and retention policy exist - and even then, only with the owner's approval for irreversible actions.

Timed 6-hour schedule (suggested)

- 0:00-0:30 Review Week 24 Day 167 card structure.
- 0:30-1:30 Threat model table in your own words.
- 1:30-2:30 Red-team run + calc hardening verification.
- 2:30-3:30 Per-week recall macro analysis; list worst weeks.
- 3:30-4:30 Generate MODEL_CARD.md; read it as if you were an external auditor.
- 4:30-5:30 Residual risks + paraphrase injections practice.
- 5:30-6:00 Commit card + day174_stats.json; link paths in the design doc.

Auditors look for missing negative results. Include the Day 170 LoRA nonsense generations in the limitations list. Include the grounded-but-wrong k-means answer. Include the PSI shift caveat from Day 173. A card that only lists victories reads as marketing and fails the Week 24 standard you already practiced.

If you translate the card to HTML or PDF later, keep the JSON as the source of truth. Manual edits to Markdown that are not re-generated from JSON will drift within a day of re-running scripts.

What to do next

Day 175 packages the HTTP service, runs 10 integration tests, and delivers the 20-question Month-6 midpoint exam.

Day 175: Integrate service + Month-6 midpoint exam

Review today: Day 174 (+1), Day 171 (+3), Day 167 (+7), Day 154 (+21).

Block 1: Service surface (30 min)

`code/capstone/service.py` exposes: - `GET /health` -> `{status: ok}` - `GET /metrics` -> eval summary from the model card - `POST /ask {question}` -> agent JSON (blocked / answer / trace)

It uses the stdlib `HTTPServer` so students need no FastAPI install for the reference path. Warm `get_agent()` once before serving traffic.

Block 2: Integration run (40 min)

```
python day175_integrate.py
```

Verified (`out/day175_integrate.log`, `day175_stats.json`): - Tests **10/10 PASS**: health, metrics, ask_calc (`12*11`->`132`), ask_inject_blocked, ask_rag, gate_recall 0.72, gate_grounded 1.0, gate_inject 1.0, gate_status PASS, retrieve_k3 (hits include `week-14.md#...`). - Exam written: `out/day175_exam.md` - **20** questions with answers covering gradient descent through differential privacy (Months 1-6 themes + capstone defences). - Exam-related retrieve smoke **5/5**. Integration **PASS**.

Block 3: Month-6 midpoint exam (use 90+ min of the day)

Work the 20 questions closed-book first, then check answers in `day175_exam.md`. Sample:

- Q10: attention output = `softmax(QK^T / sqrt(d_k)) V`
- Q14: LoRA trains low-rank adapters, not full weights
- Q16: regex/guard + allow-list + AST calc (any one)
- Q17: KV cache avoids recomputing past keys/values
- Q19/Q20: demographic parity; DP bounds single-example influence

This is the **Month 6 midpoint**, not the final cumulative exam (Week 26).

Block 4: Self-check (20 min)

1. Integration tests passed? ***10/10.***
2. Calc `12*11` answer? ***132.***
3. How many exam questions? ***20.***
4. Default service port? ***8765.***

Block 5: Practice (30 min)

Point a curl client at `/ask` with a benign course question and an injection string; save the JSON responses into your design-doc appendix. Optionally wrap `service.serve` behind uvicorn+FastAPI as a stretch - keep the stdlib path green.

Extra notes

- Metrics endpoint reads `model_card.json`; run Day 174 before Day 175 for a filled card.
- Week 26 continues write-up, rehearsal, and presentation - do not delete `code/capstone/out/`.

Going deeper

Add a `/retrieve` debug endpoint that returns top-3 ids without generation (faster CPU iteration).

Interview-style question (15 min)

"Show me it works." Reply: run the 10 tests, open the scorecard, quote recall 0.72 / injection 1.0 / p50 0.75 s, and hand the model card - not a cherry-picked chat screenshot.

Common mistakes

- Starting the server without warming the model (first-request timeouts).
- Treating the midpoint exam as optional fluff.
- Redeploying after a FAIL gate "because the demo looked fine".

Build tasks (student)

1. Run the 20-question exam closed-book; score yourself; list three weak weeks to revise before Week 26.
2. Add one integration test that `POST /ask` with an empty question returns HTTP 400.
3. Record a 90-second terminal demo: health, calc, injection block, one RAG question - no edits after recording.

The midpoint exam is cumulative by design. Months 1-3 show up as gradient descent, confusion matrices, trees, PCA, backprop, convolutions. Months 4-5 show up as attention, RAG, BM25, LoRA, ReAct, injection. Month 6 shows up as KV cache, ONNX Runtime, demographic parity, differential privacy. If a question feels unfamiliar, that is diagnostic: go back to the matching week file before the Week 26 final.

Service packaging is intentionally minimal. FastAPI is fine as a stretch, but the reference tests must pass on the stdlib server so a clean venv with `transformers`, `sentence-transformers`, `peft`, and the local `rust_score` wheel is enough. Pinning extra web frameworks without a need fails the "CPU-feasible, runnable" bar.

Release decision for Week 25: gates PASS, tests 10/10, model card present, exam filed. That is a **course checkpoint**, not a production launch. Week 26 adds narrative, video, and the cumulative final. Keep `code/capstone/out/` and `code/week-25/out/` as the evidence bundle; do not overwrite stats JSON without copying to a `freeze/` directory first.

Timed 6-hour schedule (suggested)

- 0:00-0:30 Review all gates; confirm Day 173 PASS JSON present.
- 0:30-1:30 Run integration script; fix any env issues (HF cache, rust_score import).
- 1:30-3:30 Closed-book 20-question midpoint exam; grade; note weak topics.
- 3:30-4:30 Open-book remediation on the three weakest answers using week files.
- 4:30-5:30 curl demo + optional FastAPI stretch (do not break stdlib tests).
- 5:30-6:00 Package evidence: logs, stats, card, exam, design doc checklist.

Month-6 midpoint means you are past responsible AI and deployment basics and into integration. The exam's job is to force retrieval from memory across the whole course, not to trick you. Write short answers. Where a formula is required (attention, DP intuition), prefer the canonical form from the week file over a paraphrase that drops the sqrtd_k term.

After this day you should be able to explain the capstone in five breaths: index the notes, retrieve top-3, generate with a small LM, guard tools, gate on metrics. If that sentence is missing a noun you built, revisit that day's log before Week 26's presentation.

What to do next

Week 26: write-up, demo video, cumulative review, final exam, rehearsal, presentation. Keep gates green.

Week 25 quiz (12 questions)

Answers follow each question.

1. How many chunks did Day 169/170 index from `weeks/*.md`? **Answer:** 1467.

2. Day 170 recall@3 on 25 QA pairs? **Answer:** 0.720 (18/25).

3. LoRA 60-step wall time and loss start->end? **Answer:** 14.8 s; 3.791 -> 0.961.

4. Day 171 p50 RAG+gen latency? **Answer:** 0.78 s.

5. Injection results on Day 171? **Answer:** 3/3 blocked; 0/10 false positives.

6. Rust int8 best speedup vs numpy-int8 (Day 172)? **Answer:** 50.35x (100kx256 synthetic).

7. SmolLM2 KV fp16 bytes per token? **Answer:** 23040.

8. Day 173 harness status and p50 RAG? **Answer:** PASS; 0.75 s.

9. Day 174 red-team mitigated count? **Answer:** 6/6.

10. Day 175 integration tests? **Answer:** 10/10 PASS.

11. Name the three allow-listed tools. **Answer:** calc, course_search, get_day_title.

12. What release rule fails the build even if the demo looks good? **Answer:** recall@3 drops >10% relative vs baseline, or injection block rate < 0.70 (also any budget gate FAIL).

Week 25 project checklist

- [x] Proposal + budget JSON (`day169`)
- [x] Index + LoRA baseline (`day170`)
- [x] RAG + agent + guards (`day171`)
- [x] Rust int8 + KV budget (`day172`)
- [x] Eval harness + gates + PSI (`day173`)
- [x] RAI audit + model card (`day174`)
- [x] Service tests + 20-Q midpoint exam (`day175`)